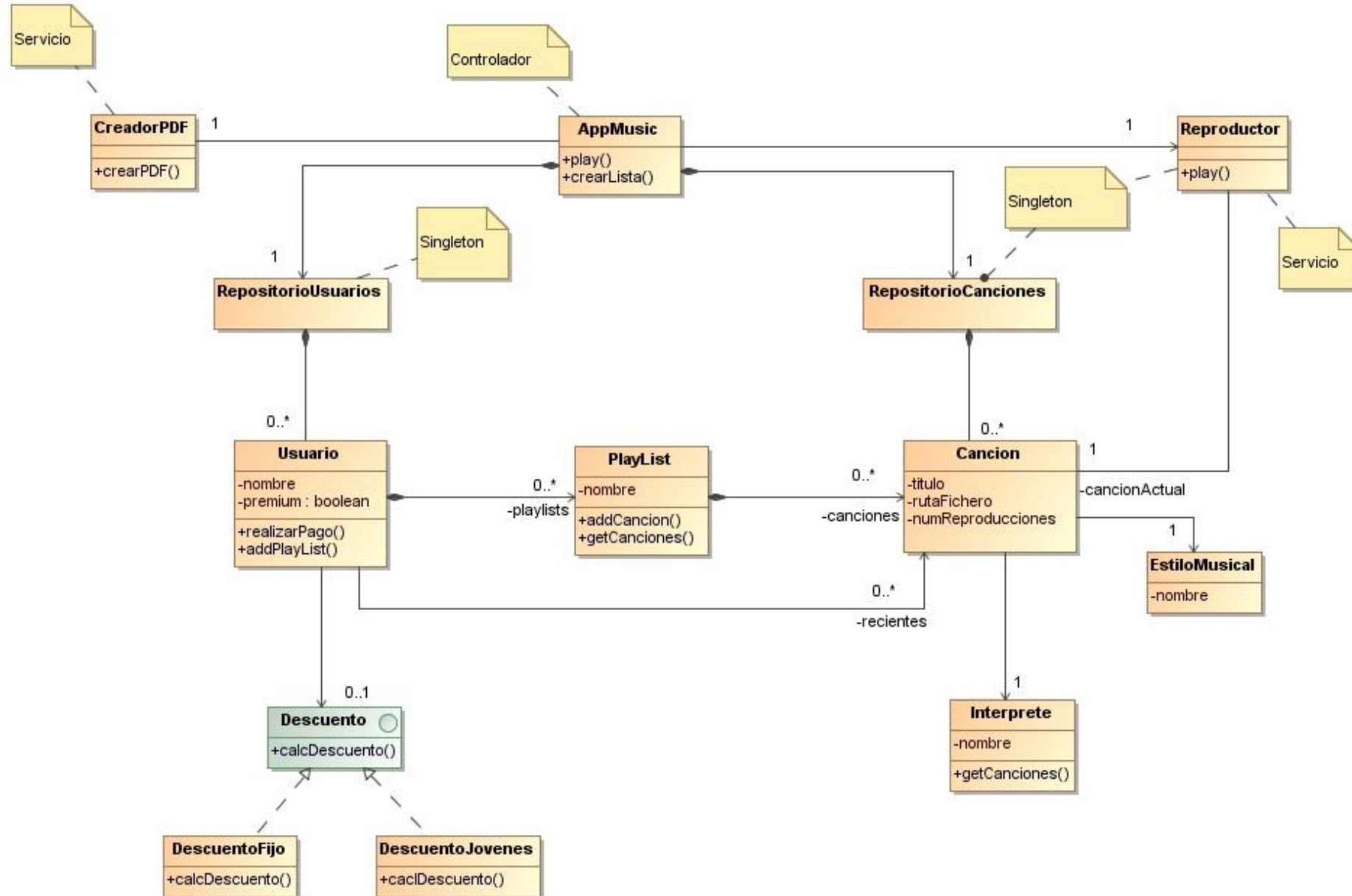


**Grado en Ingeniería Informática**

Tecnologías de Desarrollo de Software

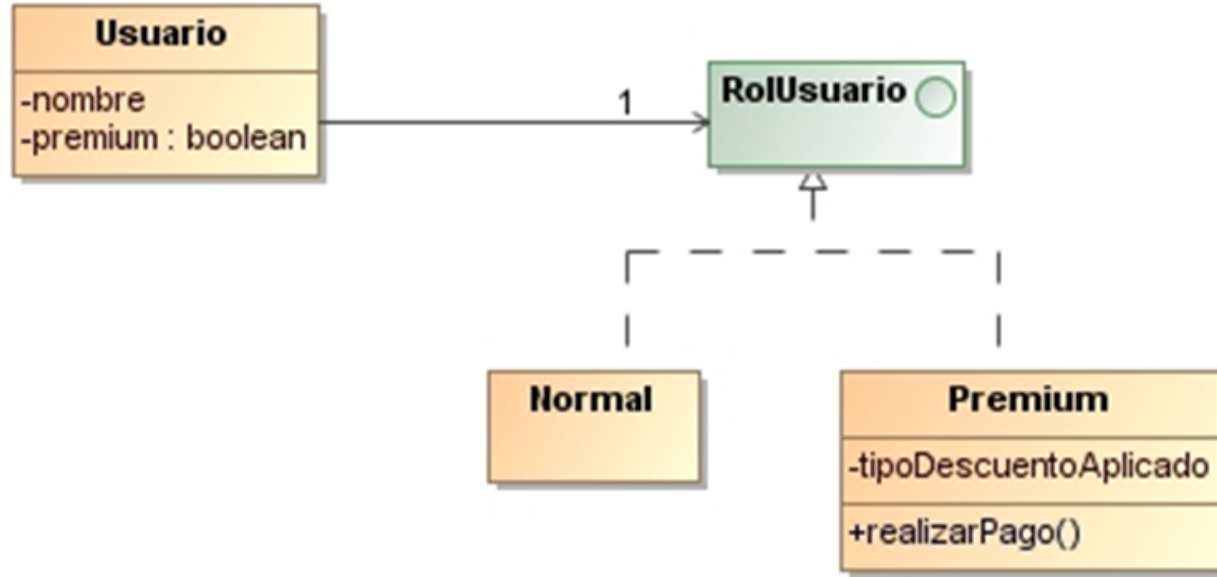
# Taller de Modelado de clases del dominio

# Diagrama de Clases del Dominio del Caso Práctico “AppMusic”



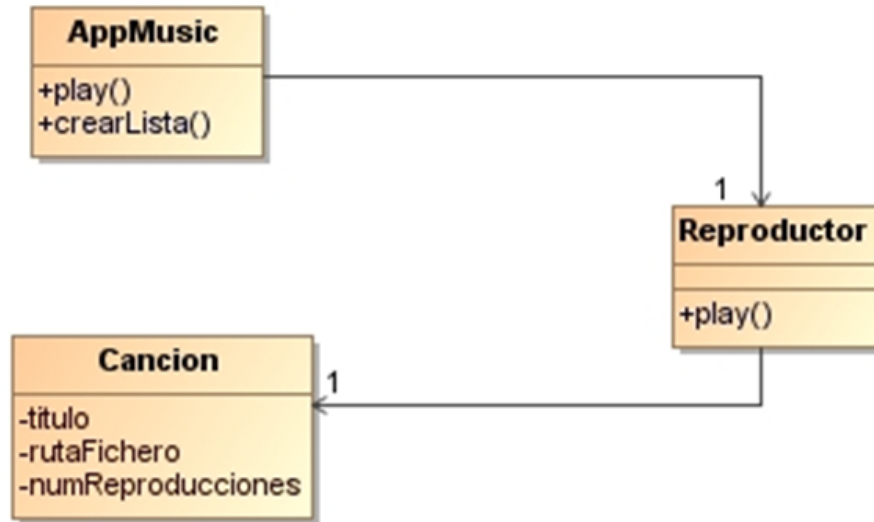
## Comentarios sobre algunas decisiones en el diseño del diagrama:

1. **Usuarios Premium.** Un usuario puede en cualquier momento convertirse en usuario Premium o dejar de serlo. Esta característica es un ejemplo de rol jugado por un usuario. La forma recomendada para representar roles de un usuario es asociando una jerarquía de roles a una clase como la que se muestra abajo.



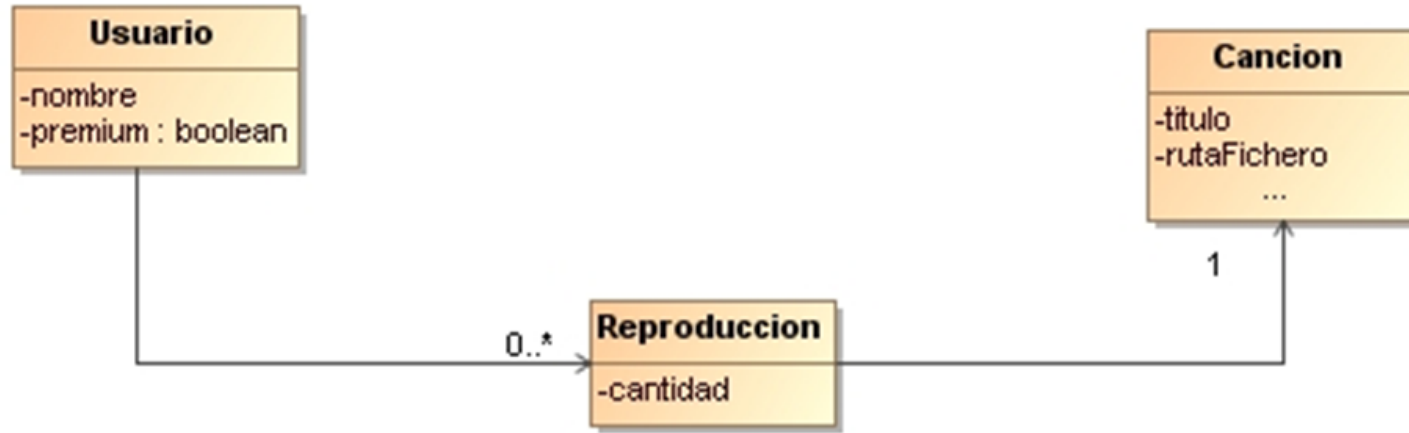
En este caso la funcionalidad que tendría la clase **Premium** y la existencia de un único rol nos ha motivado a considerar sólo una clase **Usuario**, con un atributo booleano que establece cuando un usuario es **Premium**.

2. **Clase Reproductor.** Se ha incluido la clase Reproductor que se muestra abajo. Esta clase sería un ejemplo de clase *Servicio* de DDD. En este caso la operación del negocio es reproducir canciones, la cual no debe ser parte de la clase Canción sino en una clase aparte que encapsula el acceso a la librería JavaFX.



3. **Clase CreadorPDF.** Es otra clase Servicio encargada de crear un archivo PDF con información sobre las playlists de un usuario. En el diagrama, no se ha mostrado una asociación con PlayList para evitar cruzar líneas de asociación. Encapsula el acceso al API elegida para manejar archivos PDF.
4. **Canciones más reproducidas.** Un usuario Premium puede ver las canciones más reproducidas por todos los usuarios. Por ello, la clase Cancion incluye el campo numReproducciones.

5. Nótese que si se pidiese mostrar las canciones que más veces ha reproducido **un único usuario** sería necesario añadir una clase que registre ese valor. El diagrama de abajo, la clase Reproducción es la que juega ese papel. **Esta segunda posibilidad será opcional en el caso práctico.**



6. **Canciones recientes.** Hemos considerado que no es una playlist predefinida por una razón semántica, la lista de canciones recientes no es una playlist. Podría implementarse de ese modo, pero nos parece más apropiado que usuario las registre en una lista específica.
7. **Intérprete.** Se ha incluido ya que se trata de un concepto esencial del dominio de la aplicación, pero no será necesario considerarla en la implementación de AppMusic, dado que los requisitos no lo requieren. Sería necesaria si se debiese tener un registro de las canciones de un determinado intérprete. En AppMusic, el intérprete de una canción se registrará en un campo String de la clase Canción.

8. **Estilo musical.** En el diseño de clases del dominio de una aplicación es común tener que decidir si una información se representa como un atributo de tipo primitivo o enumerado o bien una clase. En este caso, deberíamos optar por representar los estilos musicales por medio de una clase EstiloMusical teniendo en cuenta que los estilos pueden variar y llevan asociados una lista de canciones de ese estilo. Pero, al igual que con Interprete, el estilo se registrará en un campo String de Cancion.
9. **Repositorios.** Se mantendrán repositorios para los usuarios y las canciones (objetos raíz sobre los que se aplican operaciones de búsqueda, pero no para playlists). Caso de existir las clases Interprete y EstiloMusical, se podría tener un repositorio de objetos Interprete y otro de objetos EstiloMusical para realizar búsquedas más eficientes, por ejemplo, para encontrar las canciones de una categoría o de un intérprete.
10. Se han mostrado las **relaciones** más importantes; por ejemplo, AppMusic estará relacionada con casi todas las clases del dominio, pero sólo se han mostrado relaciones con los objetos servicio y repositorios. Del mismo modo, sólo se han incluido unos pocos **atributos y métodos** como es habitual en los diagramas de clase. Cada grupo deberá establecer cuidadosamente cuáles son los atributos y métodos de cada clase, y las relaciones entre clases.